

Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović^{1, 5} Christian Blum² Aleksandar Kartelj³ Sašo
Džeroski⁴ Žiga Zebec⁵

¹University of Nova Gorica, Nova Gorica, Slovenia

²Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain

³Faculty of Mathematics, University of Belgrade, Belgrade, Serbia

⁴Jožef Stefan Institute, Ljubljana, Slovenia

⁵Institute of Information Sciences (IZUM), Maribor, Slovenia

- OPTIMA 2026: XVII International Conference Optimization and Applications,
Petrovac, Montenegro –
– September 2026 –



Sofinancira
Evropska unija



SMASH
machine learning for science and humanities
postdoctoral program

Introduction

- Objects we deal with: sequences (strings) over finite alphabet Σ
 - DNA/RNA sequences over $\{A, T, G, C/U\}$
 - Proteins over 20 (canonical) amino acids: $\{A, C, D, E, P, Q \dots\}$
- **One of central tasks in computational biology:**
 - sequence comparison, find **common motifs** between sequences
 - structural similarity vs. sequence functionality
- Motifs = common **subsequences** $\rightarrow$ **Longest common subsequence problem** variants (studied 50 years already)

Longest common subsequence problem (LCSP)

Definition (LCSP)

Input: Given an arbitrary set of sequences $S = \{s_1, \dots, s_m\}$

Task: Find a longest possible subsequence s **common** for all sequences from S .

LCS: Literature & Problem Variants

- For $m = 2$: polynomially solvable, e.g. Dynamic programming
- When m arbitrary large: **NP-hard**, e.g., approximation, meta-heuristic, exact approaches

Problem-related practical variants: Repetition-free, Filled LCS problem, **Gapped LCS problem**

The gapped LCS problem (Peng and Yang, 2012)

Definition (Gapped sequence)

Given is a sequence s and a function $G_s: \{1, \dots, |s|\} \mapsto \mathbb{N}$. A pair (s, G_s) is called a gapped sequence.

(G_s : one number assigned to each position of s)

Definition (Gapped subsequence)

Sequence $\tilde{s}$ is a **gapped subsequence** of (s, G_s) iff

- $\tilde{s}$ is a **subsequence** of s
- the gap constraint G_s fulfilled resp. the *positions of embedding* $\tilde{s}$ in s :
 - **upper bounds assigned on the number of letters** between consecutive pairs of (embedding) positions

Problem definition

Example

$s = \text{AATTGC}$, $G_s(\cdot) = 1$, $\tilde{s} = \text{ATG}$

- the embedding **AATTGC** (**valid** gapped subsequence)
- the embedding **AATTGC** (**invalid**, two letter inbetween $> G_s[4] = 1$)

Definition (The multiple gapped LCS problem)

Input: Given is an arbitrary set of gapped sequences $\{(s_i, G_{s_i})\}_{i=1}^m$.

Task: Find the longest sequence $\tilde{s}$ so that

- $\tilde{s}$ is **common subsequence** of each s_i
- $\tilde{s}$ **fulfills all gap constraints** G_{s_i} ($i = 1, \dots, m$)

Literature & Motivation for VGLCSP

- Peng and Yang (2012, 2014): $m = 2$ version by **three dynamic programming** (DP) approaches
- Manea et al. (2024): complexity analysis investigated
 - **$\mathcal{NP}$ -hard** (m arbitrary)
- Djukanovic et al. (2026) the first approach for m arbitrary: **Iterative Beam search approach**

Motivation:

- **Genetics and molecular biology**
- **Time-series analysis**: in settings where events are required to occur within specified temporal delays (Lainscsek et al. (2015))

Methodology by Djukanovic et al. (2026)

- Based on the **state space graph formulation** of the problem
- **Nodes** corresp. to partial solutions and resp. subproblems to solve
 - transitions: valid extensions of partial solutions (by one letter)
- **Gap constraints**: cut-off invalid extensions (edges) among the LCS extensions
- **Many root/source nodes** in the state space
 - root state space subgraph (formulation) w.r.t. starting source (match)

Methodology: main components roughly

- Generation of high-quality solutions for a root (sub-)space generally hard $\Rightarrow$ an approx. approach for evaluation (**Beam search – limited BFS**)
- The root-state-space formulation may involve disconnected components $\Rightarrow$ needs **iterative beam search** to explore all search space

$\Rightarrow$ **Iterative Multi Source Beam Search (IMSBS) approach.**

Heuristic guidances in IMSBS

The best literature heuristic to guide BS and prioritize root node:

- ① UB_2 : *Character Frequency Alignment* score

Conclusions about IMSBS:

- efficiently executed on large synthetic instances
- Weaknesses: the performance of UB_2 improvable for large n ($= 500$) and small $|\Sigma|$ ($= 2$)

⇒ **Construction of a new heuristic guidance**

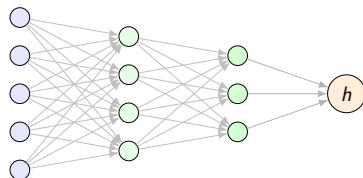
Designing advanced heuristic guidances: main idea

- We learn a **new node evaluator**: a function assigning a promise score to a partial solution

Neural network basics: MLP as a scoring function

- We use a **feed-forward multilayer perceptron** (MLP).
- Input: **compact numerical description** of the state v .
- Hidden layers transform features into nonlinear combinations.
- Output: a **single heuristic score** $h_w(v)$.

features hidden + activation funcs score



State features for VGLCS node evaluation

For a state $\mathbf{v} = (p^v, l^v)$ (p^v : vector of suffix positions encoding subproblem to be solved, l^v : length of partial solution), build a **size-independent features**:

Search progress

- normalize positions:
 $F_i^v = p_i^v / |s_i|$
- max, min, mean, std of F^v
- current length l^v

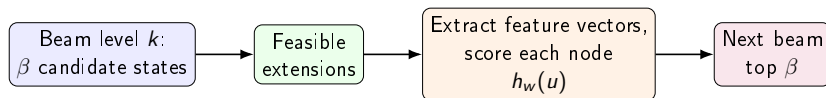
Gap flexibility

- G_i^v : gap values for the respective positions of the subproblem
- max, min, mean, std
- how restrictive future extensions are

Global context

- alphabet size $|\Sigma|$
- number of sequences m
- standardized before training

How the learned evaluator is used inside IMSBS



Important note

Learning is **offline**; during the final run the NN is only a fast scoring function.

Learning weight of NN evaluator: Why not standard supervised backpropagation?

Obstacle(s)

- Exact optimal trajectories unavailable for the general case

Solution: neuro-evolution

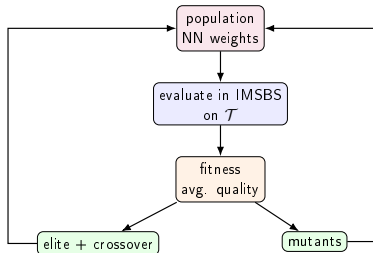
- Treat all NN weights as **one vector** w .
- Evaluate w by running (inexpensive) IMSBS on training (32) instances.
- **Fitness** = average subsequence length found

Learning weights with Biased Random Key GA

- Individual = complete NN weight vector w
- Population of weights, evolution
- ...

Parameters setting:

$n_{pop} = 20$, $n_{elite} = 1$, $n_{mutants} = 7$,
 $p_{bias} = 0.5$.



Rank-based ensemble: learned + analytical guidance

- **Preliminary tests:** the learned heuristic strong, but not uniformly best
- Hand-crafted upper bounds useful, especially in some small- m cases
- Combine both into one: **rankings** rather than scores combined

Combined rank

$$r(v) = \alpha r_{h_w^*}(v) + (1 - \alpha) r_{UB_2}(v).$$

⇒ UB_2 replaced by $r(\cdot)$ into IMSBS: **Ensemble-IMSBS approach.**

What was tuned? Experimental design

Training protocol

- IMSBS utilized $\beta = 500$, max. 100 beam iterations or 60 sec

NN design choices

- 2 or 3 hidden layers
- 5 or 10 neurons per hidden layer
- activation: tanh, ReLU, sigmoid
- different feature combinations

Selected architectures (split by diff. m), E.g.:

m	layers	act.	features
3	10-10-5	ReLU	search & gap features + $ \Sigma $
5	10-5-5	sigmoid	search & gap features
$\vdots$			

Numerical results: Random and Real benchmarks

Random benchmark

comparison [Ensemble vs. IMSBS]	wins	ties	losses
all groups (32)	20	8	4
$m = 2, 3$	10	5	1
$m = 5, 10$	10	3	3

Wilcoxon: $p = 0.009$ on all groups ($n \in \{50, 100, 200, 500\}$).

Real benchmark

comparison	wins	ties	losses
Ensemble-IMSBS vs. IMSBS (20)	12	7	1

Comparable runtime; larger gains on harder instances such as Virus.

Conclusion

- We successfully learned an improved guidance through a neuro-evolved process
 - statistically significantly better results when compared to the literature-best approach
- Noticed for $|\Sigma| = 2$: a stronger exploration/**richer feature portfolio** needed

Thank you for your attention!

Ensemble-IMSBS vs. exact DP-1 solution ($m = 2$)

$$|\Sigma| = 2$$

n	Avg. Gap
50	1.0%
100	6.5%
200	10.0%
500	24.4%

Performance degrades as
the instance size increases.

$$|\Sigma| = 4$$

n	Avg. Gap
50	0.7%
100	0.3%
200	1.1%
500	2.0%

Near-optimal performance
across all instance sizes.

**Ensemble-IMSBS stays within 2% of DP-1
for all tested $|\Sigma| = 4$ instances.**